

AgentFury

A personal AI agent that reads your email, manages your calendar and searches your own machine — built as a multi-agent runtime with two independent retrieval systems, one of them written from scratch so private documents never leave the device.

github.com/Prateek22672/agent-forge

agentfury.foliofyx.in

Desktop app — Windows & macOS

THE PROBLEM IT SOLVES

An assistant that can act, not just answer

Most assistants can talk about your work but cannot touch it. AgentFury connects to Gmail and Calendar, keeps durable memory of what matters, and — in the desktop app — searches the contents of every document on your laptop and acts on them.

Three surfaces share one backend: a web app, a Chrome extension, and an Electron desktop app whose launcher answers questions about your own files in about a second.

2	0	0.51s	670ms
independent RAG implementations	bytes of file content stored in the cloud	answer latency, measured across models	to index 51 documents into 421 passages

Two RAG systems, because they answer different questions

One remembers what you told the agent. The other reads what is already on your disk. They have different privacy constraints, so they are built differently.

SEMANTIC · SERVER

Agent memory

Facts the agent should remember across conversations, retrieved by meaning rather than wording.

- ChromaDB with all-MiniLM embeddings, running locally on the server
- A shared retrieval pipeline cleans input, packs and de-duplicates context, and strips model preamble from the reply
- Every reply exposes its retrieval and tool trace in the UI

LEXICAL · ON-DEVICE

Local file search

The text of the user's own documents, indexed and ranked entirely on their machine.

- BM25 written from scratch — no vector database, no embedding API
- Text extracted from PDF, DOCX, PPTX and XLSX using only Node's [zlib](#): a PDF page is a Flate stream, an Office file is a zip of XML
- Verified on real files: 16,225 characters from a PDF, 39,185 from a spreadsheet

Why no embeddings on the device. Embedding local files would mean uploading the text of every document to an API before the user had asked anything. BM25 needs no model, so retrieval runs offline and only the handful of passages that actually match a question are ever sent — and only at the moment one is asked. The model is instructed to answer solely from those passages, cite the file it used, and say what is missing rather than guess.

Tool-calling agents with an execution contract

Each agent is a configuration — model, system prompt, and a subset of the tool registry — compiled into a LangGraph ReAct loop. The model proposes a structured tool call, the runtime executes it and feeds the result back. Tools cover email, calendar, reminders, notes, web search and memory.

The desktop agent goes further: it turns a sentence into a file operation. "Create a folder with all my resumes and put it on the desktop" is parsed locally, resolved against the file index, and returned as a *plan* — the destination, the folder name and every file it would touch — which the user confirms before anything is written. Operations copy rather than move, and name collisions are suffixed, so a wrong guess costs a folder rather than data.

Choices that were measured, not assumed

Model chosen by benchmark	Every model on the account was timed on identical prompts, twice each. qwen3.8-27b answered correctly at 0.51s against 0.73s for the next best, so it backs every instant path, with a documented fallback if it is retired.
A dedicated answer endpoint	The launcher is judged on beating the user's patience. Routing it through the agent endpoint would resolve a conversation, run a tool loop and write to the database — all correct for a chat, all dead weight here. It gets one call to the fast model with no tools and no persistence.
Machine-written files excluded	<code>package-lock.json</code> outranked a genuine slide deck for "database schema tables" purely on npm package names. Lockfiles, minified bundles and source maps are kept out of the content index.
Reading files is opt-in	The index is off until the user agrees, and the offer appears after a real search rather than as a launch greeting. Settings reports the exact bytes held on device against the bytes held in the cloud, which is zero — shown rather than asserted.
Zero-dependency extraction	Every parser added would be third-party code running over private documents and shipping in the installer. Node already has <code>zlib</code> , which is all PDF and the Office formats actually need.

STACK

What it is built from

BACKEND	Python · FastAPI · SQLAlchemy · Pydantic · JWT sessions
AGENTS	LangChain · LangGraph ReAct runtime · custom tool registry
RETRIEVAL	ChromaDB with all-MiniLM embeddings · hand-written BM25 index · custom PDF/OOXML extractors
INFERENCE	Groq and Google Gemini, behind one provider router with key rotation and fallback
FRONTEND	React · Vite · Tailwind · PWA with push notifications
DESKTOP	Electron · global hotkey launcher · background file indexing · signed auto-update
DELIVERY	GitHub Actions building Windows and macOS installers · Vercel · Render

Scope note. Inference runs on hosted models rather than models trained here; the work is in the agent runtime, the retrieval systems and the product engineering around them. The embedding model is the one bundled with ChromaDB, running locally.